Hausa Sentiment Classification for AfriSenti

By Nshuti Shalom Silver Jr

Table of Contents

Abstract 2

1. Introduction 2
2. Related Work 3
   - 2.1 Sentiment analysis for African languages 3
   - 2.2 Sparse text representations 3
   - 2.3 Neural and multilingual pretrained models 3
   - 2.4 Gap addressed by this project 3
3. Dataset and Data Preparation 4
   - 3.1 Dataset 4
   - 3.2 Preprocessing 5
4. Methodology 5
   - 4.1 Evaluation protocol 5
   - 4.2 Classical baselines 5
   - 4.3 Recurrent models 5
   - 4.4 AfroXLMR with LoRA 5
5. Experiments and Results 6
   - 5.1 Overall comparison 6
   - 5.2 Per-class results for B4 7
   - 5.3 Confusion matrix 7

5.4 Additional model-family comparisons 8

6. Error Analysis and Discussion 10

6.1 Successful predictions 10

6.2 Failure patterns 10

7. Deployment 11

8. Limitations and Future Work 12

9. Conclusion 12

References 13

Reproducibility materials 13

Project links

GitHub Repository: https://github.com/N-SilverJr/hausa-sentiment-app
Demo Video: Hausa Sentiment Project.mp4
Live App: https://hausa-sentiment-ob5f.onrender.com

Abstract

For this project, I investigated three way sentiment classification for Hausa (a major African language which is comparatively understudied in terms of natural language processing resources). I used the Hausa portion of the AfriSenti-SemEval 2023 dataset which has positive, neutral and negative labels (Muhammad et al. 2023) then evaluated five classical baselines, three small recurrent models and one parameter-efficient fine-tuning experiment (on AfroXLMR-mini). My best system was a calibrated LinearSVC trained on a union of word and character TF-IDF features, which achieved 0.780 accuracy and 0.781 macro-F1 on the official test set outperforming my best recurrent model (0.683 accuracy, 0.681 macro-F1) and my LoRA adapted AfroXLMR model (0.602 accuracy, 0.601 macro-F1). Upon investigation of the errors I found that the strongest system had trouble distinguishing between neutral and negative messages in cases where sentiment was expressed through negation, sarcasm, code-switching or other contextual cues. I also packaged the classical and recurrent models as a Gradio application. Overall, I found that a combination of carefully normalized character and word features could be a competitive baseline for this particular dataset

while a small fine-tuning experiment failed to improve on a well-tuned sparse model.

1. Introduction

Sentiment analysis involves assigning a polarity or affective category to text. While the task has been well established for the English and other high-resource languages, there are fewer labeled data sets, fewer language-specific tools, and less representation in pretraining corpora for many African languages. This discrepancy is important both practically and methodologically: A solution that only works in English will not be helpful for the Hausa-speaking community and would not accurately process public discussion written in Hausa. Informal social-media text presents additional challenges due to spelling variation, abbreviations, code-switching, emojis, and context-dependent expressions not captured well by every modeling approach. AfriSenti-SemEval is a shared task that aims to develop a sentiment analysis resource for African languages. The task includes monolingual, multilingual and zero-shot scenarios for multiple African languages, and three sentiment classes (Muhammad et al., 2023). This project focuses on the monolingual scenario for the Hausa language and asks the question:

"**Can a carefully normalized word-and-character-based TF-IDF model provide a strong practical Hausa sentiment classifier, and how does it compare to small recurrent models and parameter-efficient adaptation of an African-language pretrained transformer?**"

I do not claim that one experiment solves the problem of Hausa sentiment classification but my goal is to set up a reproducible baseline, compare the performance of different model families on the same data split, understand the main failure modes of the approaches, and make the best model available through a web application. I implemented the preprocessing, leakage checking, model training and comparisons, error analysis, artifact packaging and Gradio deployment application. I have used AfriSenti, AfroXLMR-mini, scikit-learn, PyTorch, PEFT, and Gradio as existing resources; I have not personally developed those resources.

2. Related Work

2.1 Sentiment analysis for African languages

The AfriSenti-SemEval benchmark is significant because it assesses the performance of sentiment analyses beyond the languages that are usually found in NLP-benchmarks. Muhammad et al. (2023) introduce a shared task covering 14 languages of African origin with three sentiment categories and report the results of monolingual, multilingual and zero-shot subtasks. Although the evaluation benchmark allows comparison of different approaches, the comparative results of the shared task also highlight the strong influence of language, training setup and computational resources. Hence, testing the performance of a Hausa classifier on a Hausa test set is more informative than relying on inferences from other languages.

2.2 Sparse text representations

TF-IDF encodes a document as a distribution over terms weighted by their frequency within the document and their rarity across the corpus. The approach is simple, cheap, and often competitive on supervised text classification tasks. scikit-learn supports both word and character analyzers, including n grams of each (Pedregosa et al., 2011). The latter is particularly useful to informal and morphologically variable text since related misspellings may share subword fragments, even if their whitespace-tokenized representations differ. I test this hypothesis directly by evaluating word-only, character only, and combined representations.

2.3 Neural and multilingual pretrained models

Recurrent models can learn representations specific to tasks from sequences but require sufficient labeled data and careful optimization. I therefore include a small trainable-embedding model and two BiLSTM variants as controlled neural comparisons. Multilingual pretrained transformers such as XLM-R showcase the value of large-scale multilingual pretraining (Conneau et al., 2020). AfroXLMR adopts multilingual pretrained language modeling to African languages, including Hausa, and implements it using multilingual adaptive fine-tuning (Alabi et al., 2022). The project uses Davlan/afro-xlmr-mini, a compact version of this model that has been adapted to 17 African languages, according to its model card. Instead of updating all transformer parameters, the experiment uses Low Rank Adaptation (LoRA) to add trainable low-rank updates to selected layers while leaving most of the base parameters frozen (Hu et al., 2022).

2.4 Gap addressed by this project

While prior benchmark work established the task and pretrained-model research provided compelling alternatives, a project targeting a small deployable system also needed to address a practical question: What is the performance gain of more complex models over transparent, inexpensive features on this specific Hausa split? I address this gap through both a controlled comparison and explicit leakage handling, class-level evaluation, and qualitative error analysis.

3. Dataset and Data Preparation

3.1 Dataset

I use the Hausa portion of the AfriSenti-SemEval 2023 shared-task release. I keep the official three-way labels:

Split	Examples
Train	14,172

Original development set	2,677
Development set after train-overlap removal	2,462
Official test set	5,303
Development rows overlapping train	215

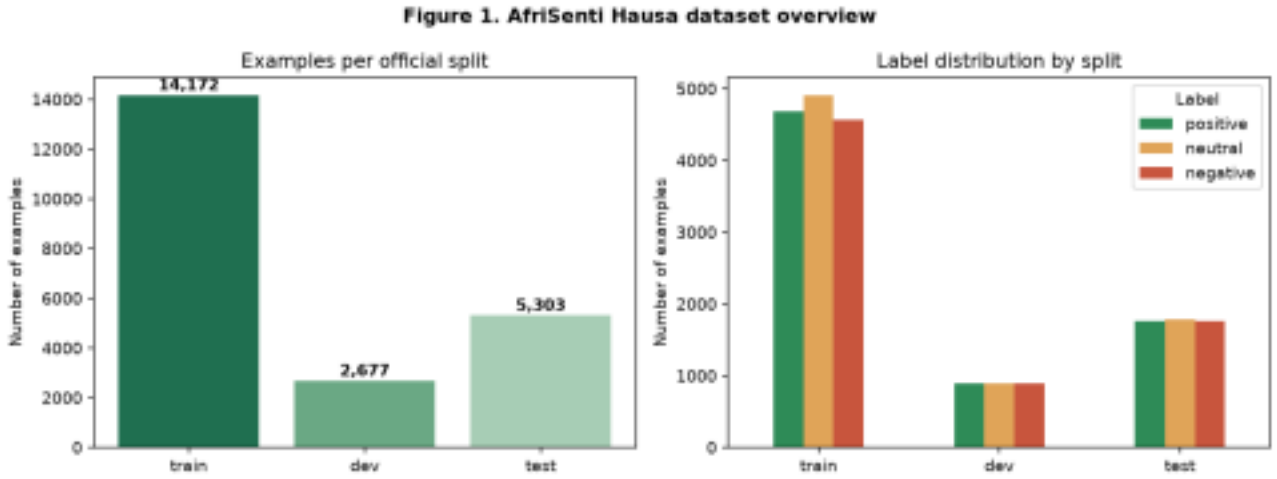


Figure 1. The official split sizes and the positive, neutral, and negative label counts are broadly balanced across the three splits.

The labels are positive, neutral, and negative. I kept the official test set aside for reporting results and used dev_noleak for model selection. I created this development set by excluding rows that appeared in the training set because choosing the model based on examples also included in the training would make little sense. I chose this dataset because it is a task-specific human-labeled benchmark created for African language sentiment evaluation. The difficulty inherent in the examples reflects what I was looking for in terms of social-media text to analyze. Namely, many of them are short, informal, ambiguous, or dependent on cultural or conversational context.

3.2 Preprocessing

I designed the preprocessing pipeline to normalize noise, leaving Hausa characters intact:

1. Normalize Unicode text to NFC.

2. Replace control characters with spaces.

3. Find URLs and user mentions.

4. Collapse whitespace and trim leading/trailing whitespace.

5. For the bag-of-words representation, remove detected URL and mention placeholders.

I apply the same text_bow transformation during training and in the application. I did not impose an

English stop-word list or stemming rule, because I did not want to assume that English preprocessing is appropriate for Hausa. For the recurrent model, I truncate sequences to 64 whitespace tokens and pad shorter sequences. I learn its vocabulary from training text, keeping tokens with frequency at least two and reserving IDs for padding and unknown tokens.

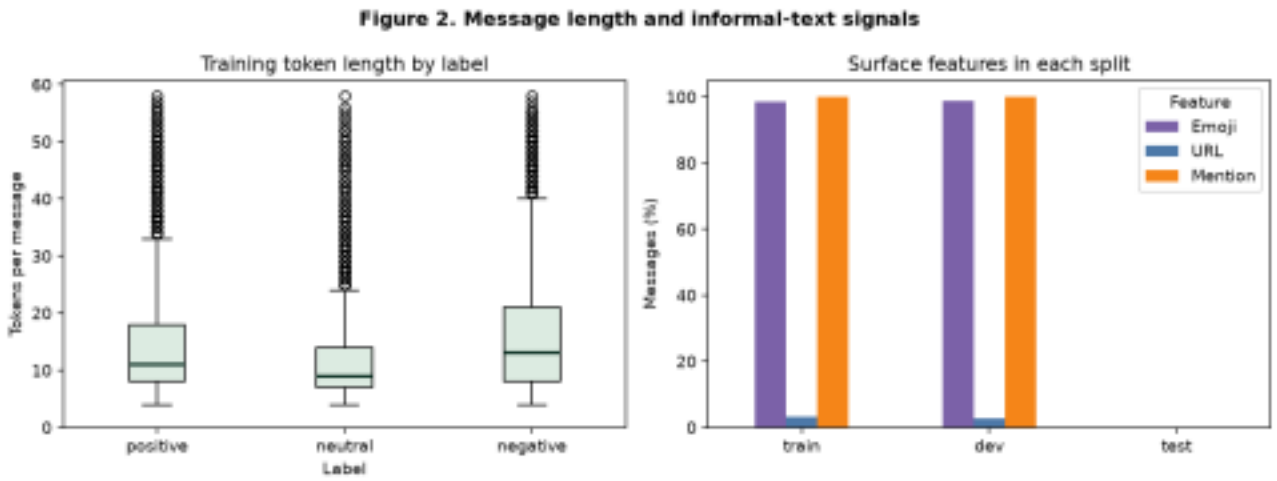


Figure 2. Message length varies across sentiment classes, while the split-level surface-feature rates show the informal social-media character of the data.

4. Methodology

4.1 Evaluation protocol

I use the same training split, leakage-controlled development split and official test split for all model families. I use random seed 42 where applicable. I select configurations by development macro-F1, as this puts equal emphasis on all classes. I report accuracy, per-class F1, weighted F1 and confusion matrices. Finally, having selected configurations by dev_noleak, I evaluate the chosen model once on the official test set.

4.2 Classical baselines

I compare the following classifiers: the majority-class dummy classifier; word TF-IDF + logistic regression; word TF-IDF + calibrated LinearSVC; character TF-IDF + logistic regression; combined word and character TF-IDF + calibrated LinearSVC; and combined word and character TF-IDF + ComplementNB. My word vectorizer uses word unigrams and bigrams, min_df=2, max_df=0.95, sublinear TF scaling. My character vectorizer uses character n-grams of lengths 3-5, same min/max DF and TF scaling. The combined features are formed by passing them through FeatureUnion. The baseline B4 uses these combined features with LinearSVC(C=1.0), calibrated using three-fold cross-validation via CalibratedClassifierCV to produce the probability-like confidence values requested for the application.

4.3 Recurrent models

For the recurrent experiments, I create a vocabulary from the training split, use a 128 dimensional trainable embedding layer and 128 dimensional hidden configuration, R1 is a masked mean pooling over the embeddings, R2 uses a BiLSTM followed by mean pooling, while R3 is a BiLSTM with attention pooling. The models are optimized using AdamW and cross-entropy loss, with batch size of 128, and trained up to eight epochs with early stopping after two consecutive non-improving development evaluations. The best performing recurrent configuration was selected according to development macro-F1 score.

4.4 AfroXLMR with LoRA

For the transformer experiment, I start from Davlan/afro-xlmr-mini, tokenize the raw text with maximum sequence length of 96 and fine-tunes a three-class sequence-classification head. I apply LoRA to the query and value modules with rank of 8, alpha of 16, and dropout of 0.1. I trained the model with batch size of 16, gradient accumulation of 2, learning rate 2e-4, weight decay 0.01, and three epochs. Only 296,451 out of 117,938,310 parameters are trainable. This parameter-efficient setup is substantially lighter than training the whole model. However, I treat the short run and compact model as one experiment rather than as an exhaustive transformer optimization.

5. Experiments and Results

5.1 Overall comparison

System	Development macro F1	Test accuracy	Test macro F1
Majority dummy	0.173	0.337	0.168
Word TF-IDF + LogReg	0.725	0.748	0.749
Word TF-IDF + LinearSVC	0.727	0.752	0.753
Character TF-IDF + LogReg	0.760	0.775	0.776
Word + character TF-IDF + ComplementNB	0.716	0.742	0.739
Word + character TF-IDF + calibrated LinearSVC (B4)	**0.762**	**0.780**	**0.781**
Trainable embedding + mean pooling (R1)	0.682	0.683	0.681
BiLSTM + mean pooling (R2)	0.668	Not retained	Not retained

BiLSTM + attention pooling (R3)	0.662	Not retained	Not retained

System	Development macro F1	Test accuracy	Test macro F1
AfroXLMR-mini + LoRA	0.662	0.602	0.601

By comparing the classical systems I see a progression, with word-only features being useful but the addition of character features improving the test macro-F1 from 0.749 to 0.776. Taking the combined features performs best, though the gain over the character features alone is relatively small, and I interpret this as evidence that subword patterns contain most of the useful signal while word n-grams provide only limited additional context. B4 improves on my majority baseline by 0.612 in test macro-F1 and on the best word-only baseline by 0.028. It also surpasses R1 by 0.099 macro-F1 and the LoRA experiment by 0.179 macro-F1. I do not interpret these comparisons as evidence that sparse approaches are universally superior: I used one compact transformer, trained for only three epochs, and performed limited tuning. What I can say is that the addition of complexity did not produce an improvement over the configurations I tried.

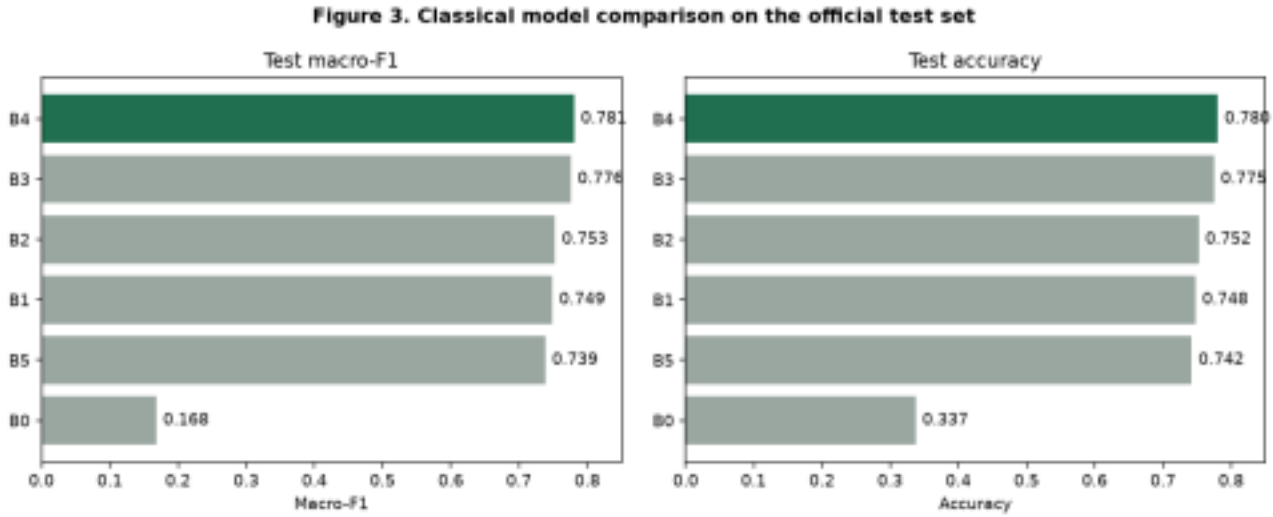


Figure 3. B4 is the strongest classical system on both test macro-F1 and accuracy; the green bar identifies the selected model.

5.2 Per-class results for B4

Class	Precision	Recall	F1	Support
Positive	0.862	0.844	0.853	1,755
Neutral	0.722	0.767	0.744	1,789

Negative	0.762	0.730	0.746	1,759
Macro average	**0.782**	**0.780**	**0.781**	**5,303**

In my results, the positive sentiment is the easiest class by F1. This agrees with my error analysis: explicit praise and goodwill expressions often have strong lexical evidence. Neutral and negative sentiment are substantially closer and negative examples have the lowest recall.

Figure 6. B4 per-class performance

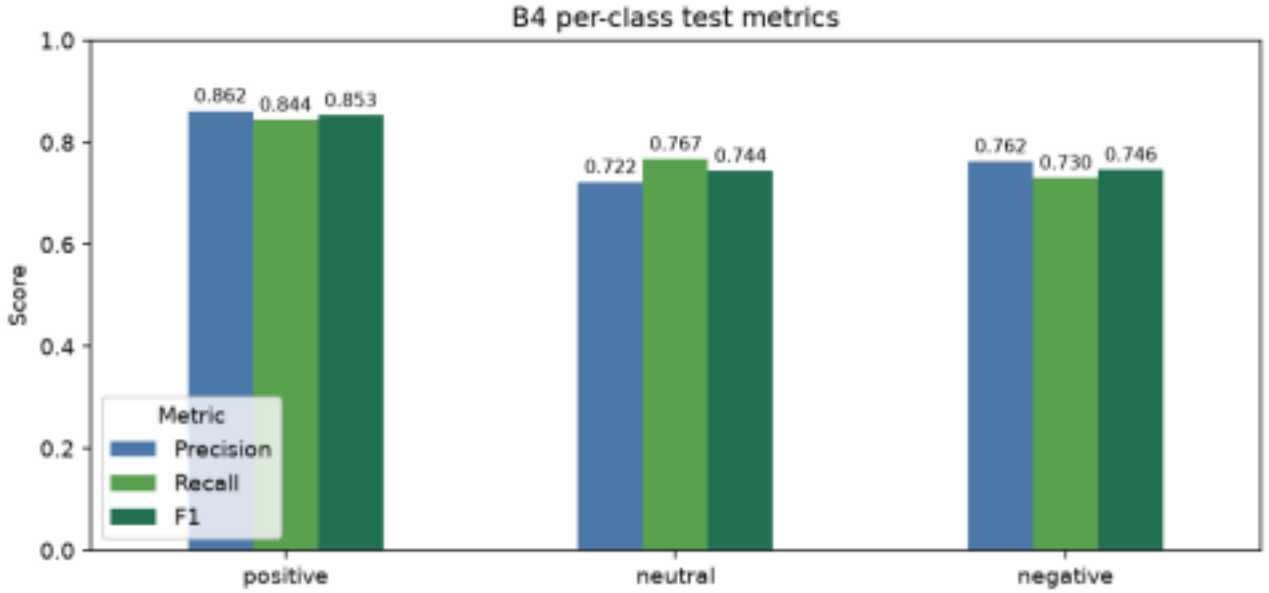


Figure 6. B4 performs best on positive sentiment and has lower, more closely matched scores for neutral and negative sentiment.

5.3 Confusion matrix

Rows are gold labels and columns are predictions; label order is positive, neutral, negative.

Gold \ Predicted	Positive	Neutral	Negative
Positive	1,481	176	98
Neutral	115	1,372	302
Negative	123	352	1,284

I found that the largest number of confusion came in negative examples predicted as neutral (352 examples), followed by the neutral examples predicted as negative (302). In other words, B4 is not primarily failing on the positive class; it has difficulty in distinguishing weak, implicit, or contextual polarity from neutral language.

Figure 7. B4 confusion matrix

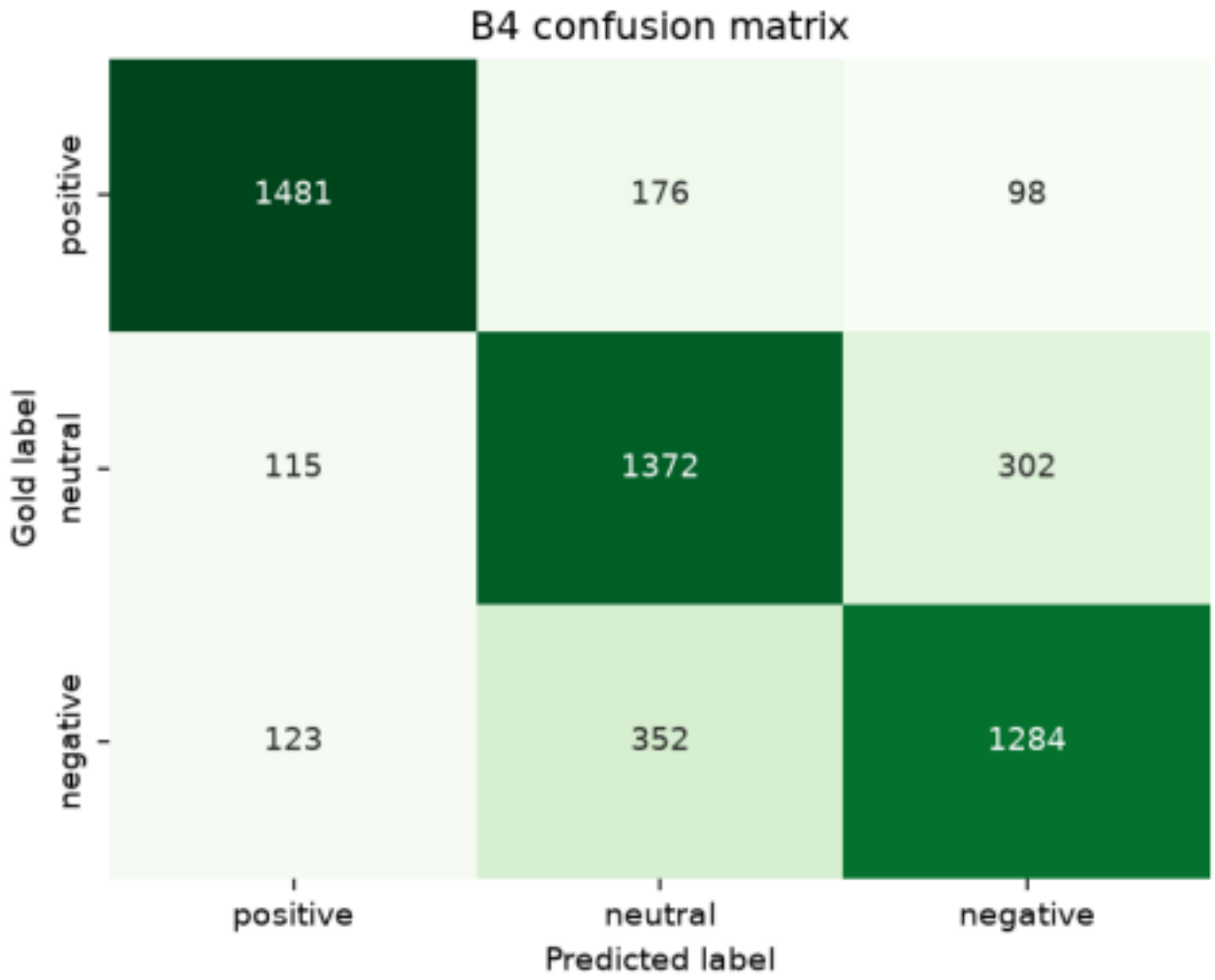


Figure 7. Most errors occur between neutral and negative messages, matching the largest off diagonal counts in the matrix.

5.4 Additional model-family comparisons

Figure 4. Recurrent-model comparison

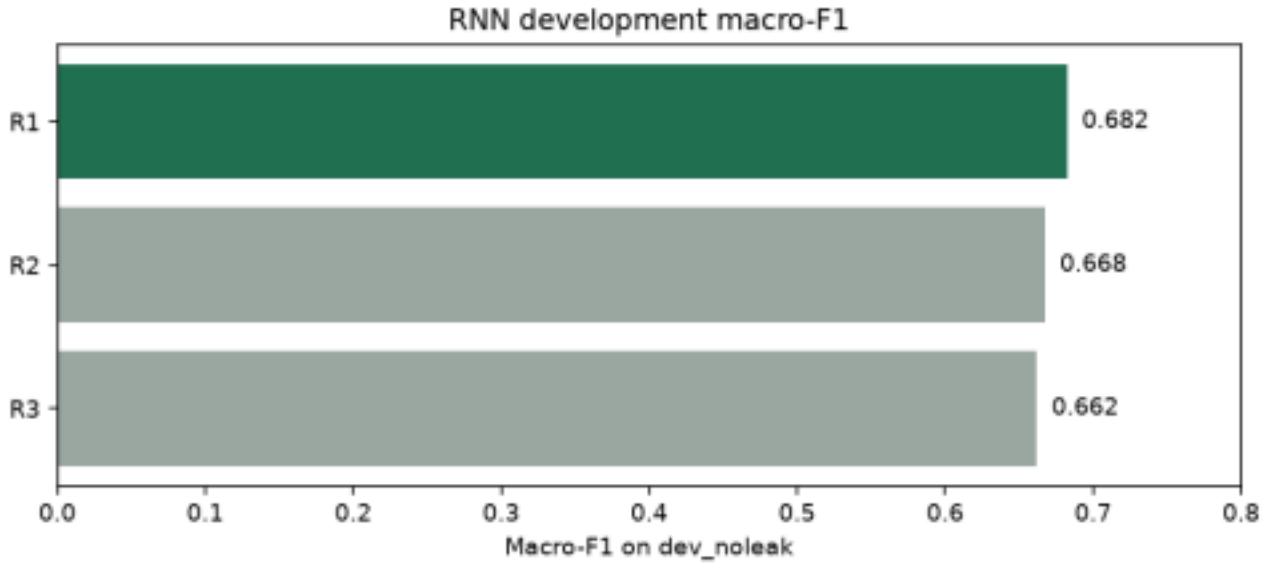


Figure 4. The simple trainable-embedding model (R1) is the strongest of the three recurrent configurations on dev_noleak.

Figure 5. LoRA development and test metrics

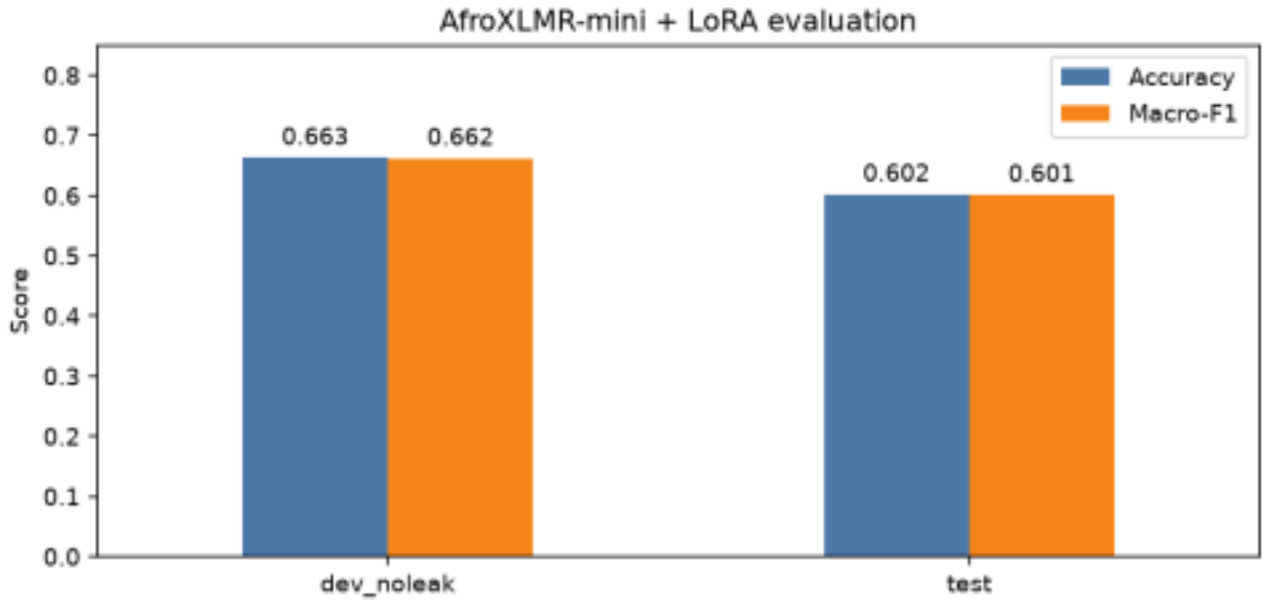


Figure 5. The compact AfroXLMR-mini LoRA run loses performance from dev_noleak to the official test set under the tested configuration.

6. Error Analysis and Discussion

6.1 Successful predictions

In my findings, I have identified several high-confidence correctly predicted positive, negative, and neutral instances, such as masha allah allah yakara basira, mtsww karyar banza, and saura nawa nima. These examples represent the three types of cases where my approach works correctly. The

first case is the use of a known positive expression (positive sentiment). The second case is using a word or phrase that contains an insult or accusation (negative sentiment). The third case is an informative short sentence with no overt polarity (neutral). I have noticed that in cases where informal spelling is involved, the character-based approach helps to detect positive instances. Fragments of words that are related to each other may also contribute to the features. This might explain why my character-only model has a higher accuracy than word-only models and why the union of both approaches performs better.

6.2 Failure patterns

The largest error directions and likely causes are:

Error direction	Count	Interpretation
Negative -> neutral	352	Implied criticism or insufficiently distinctive negative cues
Neutral -> negative	302	Informal wording or rhetorical phrasing interpreted as negative
Positive -> neutral	176	Positive intent expressed indirectly or with weak lexical cues
Negative -> positive	123	Religious or goodwill vocabulary conflicting with negative context
Neutral -> positive	115	Short messages with ambiguous positive-looking tokens

Representative failures include a positive message predicted negative due to negative-looking words, negative wish for failure predicted positive due to religious vocabulary, and short informal messages whose intended meaning is difficult to recover without conversation context. Interpretations are hypotheses, not new annotations, but are consistent with the confusion matrix. From these examples, I relate numerical results to limitations of sparse models: TF-IDF finds superficial patterns, not compositional meaning or pragmatic intent. In my experiments, it could not reliably resolve negation scope, sarcasm, code-switching, or whether a religious expression was praise, irony, or part of a negative statement. A larger contextual model would help but would need suitable tokenizer, pretraining, adaptation schedule and fine-tuning for Hausa.

7. Deployment

I have saved the selected B4 model as models/classical_best.joblib and the best recurrent model as models/rnn_best.pt for comparison. In app/app.py, I load both artifacts, apply the same normalization used during training, and return each model's predicted label and class scores. I have

also added warnings for model disagreement and low confidence. I have configured the application for Render with render.yaml. It has Python 3.13, installs app/requirements.txt, starts with python app/app.py, binds to Render's PORT environment variable and has / as its health check. The live system is at https://hausa-sentiment ob5f.onrender.com.The interface also handles obvious out-of-domain and malformed inputs. It ignores empty or noise-only messages, limits input to 2,000 characters, and displays an advisory for non-Latin scripts or longer messages that have no common Hausa cues. These checks are advisory rather than a claim of automatic language identification: short Hausa messages may not have the heuristic cues, and Hausa social-media text may have code-switching. The classifier can still return a result in those cases, but the user is warned to treat it as an out-of-domain prediction. This is a demonstration system not a production moderation service and predictions should be treated as model outputs rather than as judgments about a speaker or message.

8. Limitations and Future Work

Data limitations

I performed the study on a finite labeled sample of social-media text, which might not necessarily reflect all the dialects, regions, domains, age groups, or writing styles in Hausa. The labels narrow down possible sentiment categories, while many real-world messages contain mixed, sarcastic, or ambivalent expressions. The development overlap issue is also a concern, which makes me think that dataset auditing is required; future work should attempt to trace the origins and deduplication policy more systematically and report whether the leakage of near-duplicates (rather than exact duplicates) remains a concern.

Modeling limitations

My experiments are comparing the results of the mostly useful but limited configurations. The recurrent models are using the reduced vocabulary and reduced maximum length. The transformer experiment is utilizing a single compact AfroXLMR model, 3 epoch training and only 1 configuration for LoRA fine-tuning, which is not an exhaustive search over hyperparameters. Moreover, we did not leverage any external Hausa lexicon, pretrained word embeddings, data augmentation, or ensembling. The classical confidence values are not necessarily calibrated probabilities; a separate calibration study is necessary to obtain theoretically valid confidence intervals.

Evaluation limitations

I report one official test split and do not give confidence intervals or repeated-seed variance; a better evaluation would use repeated seeds or bootstrap intervals, keep the official test set as a final holdout throughout development, and report calibration metrics. Human review of error categories

would make the qualitative analysis more reliable.

Future work

If I continued this work, I would explore more AfroXLMR fine-tuning, test additional LoRA settings, try Hausa-specific continued pretraining, pretrained subword or character embeddings, and data augmentation targeted at spelling variation and code-switching. A hierarchical or context-aware model could handle negation and conversational context. Evaluation should include robustness to orthographic variation, dialect differences, adversarial spelling, and domain shift; finally, a user-facing system should include uncertainty thresholds, monitoring, model cards, and human review for high-impact use.

9. Conclusion

I have created and evaluated a reproducible Hausa sentiment classification pipeline. On the AfriSenti Hausa test set, my calibrated word-plus-character TF-IDF LinearSVC was the best performing method of those I tested, achieving an accuracy of 0.780 and a macro-F1 of 0.781. This was substantially better than my small recurrent models and compact LoRA-adapted AfroXLMR experiment under the selected settings. It was also practically valuable since the winning model was fast, small enough to package conveniently, interpretable at the feature-family level, and already integrated into a web application. This project also taught me that model sophistication and model quality are not directly related. Character and word features captured an exceptionally rich lexical signal in this dataset, and my remaining errors were predominantly contextual: neutral versus negative sentiment, implicit meaning, negation, sarcasm, code switching, and culturally dependent expressions. If I were to extend the project, I would combine stronger contextual modeling with improved data analysis and evaluation rather than simply add a larger architecture.

Reproducibility materials

The repository contains the processed data, experiment registries, model artifacts, and error-analysis outputs used in this report:

results/data_summary.json
results/classical_registry.json
results/rnn_registry.json
results/transformer_lora_metrics.json
results/error_analysis_summary.json
notebooks/02_classical_baseline.ipynb
notebooks/03_rnn.ipynb
notebooks/04_transformer_lora.ipynb
notebooks/05_error_analysis.py

References

Alabi, J. O., Adelani, D. I., Mosbach, M., & Klakow, D. (2022). Adapting pre-trained language models to African languages via multilingual adaptive fine-tuning. *Proceedings of COLING 2022*, 4336-4349. https://aclanthology.org/2022.coling-1.382/

Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L., & Stoyanov, V. (2020). Unsupervised cross-lingual representation learning at scale. *Proceedings of ACL 2020*, 8440-8451. https://aclanthology.org/2020.acl-main.747/

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low rank adaptation of large language models. *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2106.09685

Muhammad, S. H., Abdulmumin, I., Yimam, S. M., Adelani, D. I., Ahmad, I. S., Ousidhoum, N., Ayele, A., Mohammad, S. M., Beloucif, M., & Ruder, S. (2023). SemEval-2023 Task 12: Sentiment analysis for African languages (AfriSenti-SemEval). *Proceedings of the 17th International Workshop on Semantic Evaluation (SemEval-2023)*. https://arxiv.org/abs/2304.06845

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830. https://jmlr.org/papers/v12/pedregosa11a.html

PyTorch Contributors. (n.d.). *PyTorch documentation*. https://pytorch.org/docs/stable/

Render. (n.d.). *Render documentation: Web services*. https://render.com/docs/web-services

The Gradio Team. (n.d.). *Gradio documentation*. https://www.gradio.app/docs